

먼저 용어부터 — 창고 · 영수증 · 장부 · 티켓북

예시: 던전에서 몬스터 '고블린'을 스폰하려면 Goblin 프리팹(에셋)이 필요하다

에셋 = 재료

프리팹·텍스처 같은 실제 데이터.
평소엔 디스크의 '창고(번들)'에
압축되어 잠들어 있음.

로드 = 창고에서 꺼내기

창고 → 메모리로 꺼내오는 일.
디스크를 읽어야 해서 시간이 걸림
(그래서 async로 기다림).

핸들 = 영수증

꺼낼 때 Unity가 발급하는 증표.
"반납(Release)"해야 메모리에서 내려감.
반납 깜빡 → 누수 / 너무 일찍 → 핑크 텍스처

레지스트리 = 장부

영수증을 전부 보관하는 유일한 곳.
에셋마다 '사용자 수'를 세고,
0이 되는 순간에만 진짜 반납.

스코프 = 티켓북

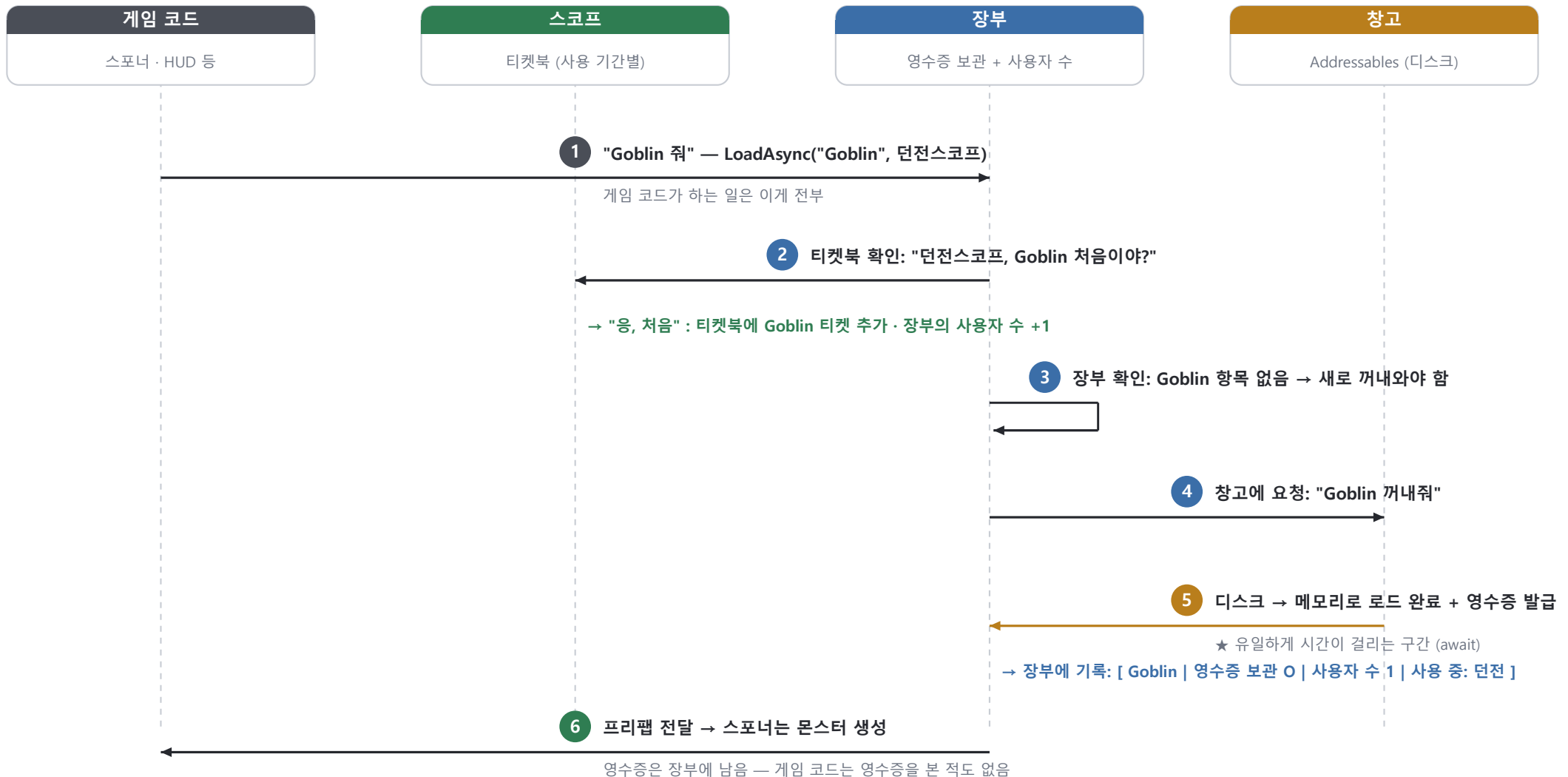
"언제까지 쓸 건지" 기간 단위 묶음.
(던전 동안 / 씬 동안 / 게임 내내)
기간이 끝나면 티켓 전부 자동 반납.

핵심 한 줄

게임 코드는 영수증(핸들)을 직접 만지지 않는다 — 장부가 보관하고, 티켓북(스코프)이 기간이 끝나면 알아서 반납한다

장면 1 — 던전 입장, 스폰어가 처음으로 "Goblin 쥬"

LoadAsync("Goblin", 던전스코프) 한 줄을 호출했을 때, 내부에서 벌어지는 일



이 장면의 포인트

- 게임 코드의 책임은 ① 하나뿐. 반납 예약(티켓)은 ②에서 자동으로 끝남
- 시간이 걸리는 건 ⑤(디스크 읽기)뿐 — 나머지는 전부 딕셔너리 조회 수준

⑤ 직후 장부 상태

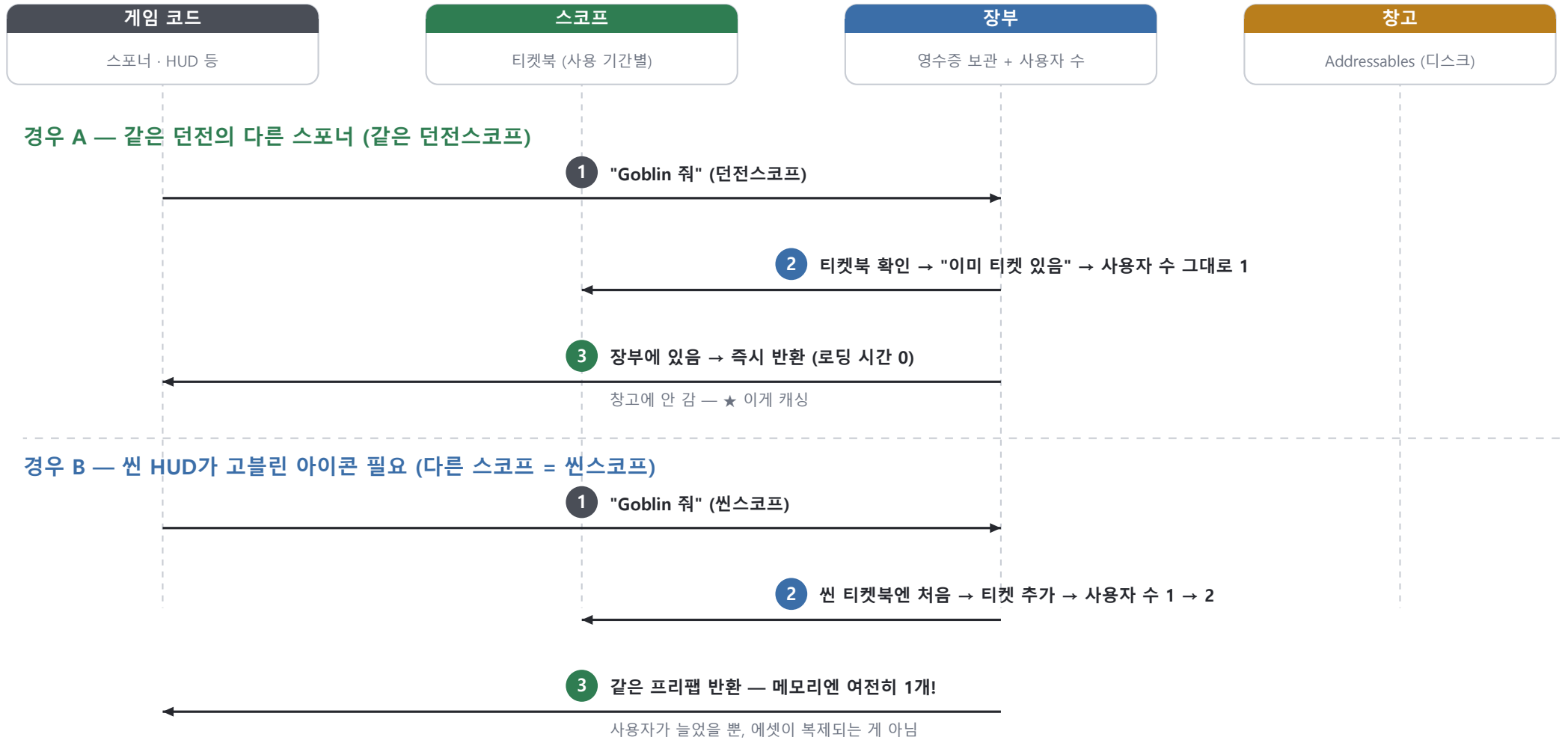
영수증: 보관 중 (O)

사용자 수: 1

사용 중: [던전]

장면 2 — 5초 뒤, 여기저기서 또 "Goblin 줘"

이미 메모리에 있으니 참고(디스크)에는 다시 안 간다 — 장부에서 바로 꺼내준다



이 장면의 포인트

- 몇 번을 다시 불러도 디스크는 한 번만 읽음 — 이후는 전부 공짜
- '사용자 수'는 스코프(기간) 단위로만 올라감 — 같은 스코프에서 100번 불러도 1

경우 B 이후 장부 상태

영수증: 보관 중 (O)

사용자 수: 2

사용 중: [던전, 씬]

장면 3 — 던전 클리어, 그리고 씬 전환

아무도 개별 Release를 호출하지 않는다 — 시간이 끝난 티켓북이 자동으로 정산한다



같은 상황, 기존 방식이었다면?

기존 — 2버킷 + 수동 관리

- 던전에서 쓴 Goblin도 '씬 버킷' 소속 →
던전이 끝나도 씬 끝까지 메모리 상주
- 개별 반납하려면 코드마다 Release를 기억해야 함
→ 하나만 깜빡해도 누수 (실제로 HP바 누수 겪음)

새 방식 — 스코프 자동 정산

- 던전 것은 던전 끝나는 순간 자동 반납
- 씬이 같이 쓰던 건 사용자 수 덕분에 안전하게 생존
- 반납을 '기억'하는 코드가 아예 없음
→ 깜빡할 수가 없는 구조 = 누수가 구조적으로 차단